Institución Universitaria

INGENIERIA DE SISTEMAS

ARQUITECTURA DE SOFTWARE I

DISEÑO SOSTENIBLE Y PRINCIPIOS SOLID

MARIA FERNANDA MUÑOZ JARAMILLO

PRESENTADO A:

MANUEL ALEJANDRO DOMINGUEZ GUERRERO

MEDELLÍN

2026

- Fase 1: Diagnostico SOLID y métricas Baseline

1. Matriz de violaciones SOLID

- SRP: la clase GestorCitasOdontologicas tiene varias responsabilidades distintas: calcula tarifas y copagos basado en complejas reglas de negocio, maneja las políticas de cancelación con multas que varian,se encarga de la persistencia comunicándose directamente con la base de datos y acumula métricas internas de estado

Evidencia: toda esta lógica está concentrada en los métodos AgendarCita() y CancelarCita()

- OCP: el cálculo de copagos emplea una serie de condiciones encadenadas, (if-else, if para cada tipo de especialidad == 1,2,3,4 y tipoconvenio == 2,3) si la clínica quiere añadir una nueva especialidad o un nuevo convenio, será necesario cambiar el código fuente de esta clase, lo que infringe el principio de estar abierto a la extensión, pero cerrado a la modificación

- LSP: A pesar de que este fragmento no muestra una jerarquía de clases complicada, se puede notar una fuerte dependencia en tipos específicos de entidades y el uso de excepciones generales en lugar de excepciones específicas del dominio que podrían ser reemplazadas o manejadas de manera adecuada por las clases que heredan

- ISP: las clases de apoyo como NotificacionServicio reúnen diferentes comportamientos dispares en un solo contrato o método grande, forzando que ambos canales de comunicación se activen juntos (correo y SMS) lo que impide que un cliente utilice solo la interfaz que realmente requiere

- DIP: la clase superior GestorCitasOdontologicas crea de forma directa y estricta sus dependencias de infraestructura a través de la palabra clave: new

→ Private sqlServerEjecutor _db = new SqlServerEjecutor()

→ Private NotificacionServicio _notificador = new NotificacionServicio();

Depende de implementaciones específicas de bajo nivel en vez de en abstracciones, lo que dificulta la separación del código para realizar pruebas unitarias

2. Linea Base cuantitativa (Baseline)

- LCOM96b: mide que tan unidos están los métodos de una clase en función de las variables que utilizan. En el caso de una clase que realiza diversas funciones, los métodos no utilizan las mismas variables, lo que indica una cohesión baja

Análisis: la clase GestorCitasOdontologicas combina elementos de lógica de precios, fechas, bases de datos y avisos. Esto origina un valor de LCOM96b bastante alto (cercano a 1), lo que señala una cohesión deficiente. La clase abarca múltiples tareas que no están relacionadas entre si

- Acoplamiento Ca y Ce

→ Acoplamiento Aferente (Ca): clases externas dependen de esta clase
Valor Ca = 1, solo la clase de arranque o pruebas la llama directamente

→ Acoplamiento Eferente (Ce): de cuantas clases externas depende esta clase para funcionar
Valor Ce = 5, depende directamente de SqlServerEjecutor, NotificaionServicio, Cita, Paciente y Odontólogo

- Inestabilidad (I), Abstracción (A), Distancia (D)

Inestabilidad (I):

$$I = \frac{Ce}{Ca + Ce} = \frac{5}{1+5} = \frac{5}{6} \approx 0.83$$

Significa que la clase es muy inestable y débil frente a cualquier alteración externa

Abstracción (A): A=0

Es cero debido a que GestorCitasOdontologicas es una clase completamente concreta, no utiliza ni interfaces ni clases abstractas

Distancia (D):

$$D = |A + I| - 1 = |0 + 0.83| - 1 = |-0.17| = 0.17$$

Al establecer A=0, I=0.83, la clase se sitúa en un área que puede ser susceptible a la rigidez o al desajuste en la medida de las métricas arquitectónicas

- Fase 3: Recalculo de métricas e impacto cuantitativo

Métrica	Estado inicial	Estado final	Impacto y beneficio
LCOM96b	Alto (=1.0) debido a la mezcla de bases de datos, reglas de precios y notificaciones en un solo lugar	Bajo (=0.0 a 0.2 por clase especializada)	Cada componente (calculadorTarifas, GestorCitasRefactorizado) asume una única responsabilidad, maximizando la cohesión interna
Ca	1 Únicamente invoca por la rutina de arranque	1 o distribuido en las clases consumidoras de la interfaz	Se mantiene el control de entrada, pero ahora los módulos interactúan mediante contratos desacoplados
Ce	5 (sqlServerEjecutor, NotificacionServicio, Cita, Paciente, Odontólogo)	Reducido drásticamente a nivel de clases concretas de infraestructura	El núcleo de negocio ya no depende de implementaciones de bajo nivel, sino de interfaces, cumpliendo así con el principio DIP
Inestabilidad (I)	=0.83 (altamente inestable y vulnerable ante modificaciones externas)	Balanceada y controlada por capas	El núcleo de la lógica de negocio se aísla, volviéndose estable ante cambios en tecnologías externas cumpliendo OCP
Abstracción (A)	0 (clase totalmente concreta, sin contratos abstractos ni interfaces)	Mayor a 0 (incorporación explicita de interfaces y clases abstractas)	Se establecen contratos formales que permiten extender el comportamiento del software sin alterar el código existente
Distancia a la secuencia principal (D)	0.17 (posicionamiento cercano a zonas propensas a la rigidez y fragilidad)	Cercana a 0 (equilibrio optimo en la secuencia principal)	El diseño alcanza el balance ideal entre abstracción e inestabilidad, logrando una arquitectura verdaderamente sostenible

Análisis

Optimización de la cohesión (LCOM96b):al dividir la god class en clases que tienen objetivos específicos, se eliminaron los métodos excesivos que unían funciones diferentes. Cada clase se ocupa únicamente de las variables y atributos que son realmente necesarios para su dominio

Desacoplamiento estructural (Ce): al liminar la creación directa con la palabra clave new para servicios externos, se rompió la conexión rígida con la base de datos y los servicios de notificación, lo que permitió una inyección de dependencias más limpia

Equilibrio arquitectónico (D): la reducción en la distancia hacia la secuencia principal indica un diseño avanzado, separado y adaptable a cambios, cumpliendo con los niveles de calidad esperados